

Business Case: Target SQL

Submitted by: Aravinth Baskar

Batch: DSML Dec25(2) Beginner – (MWF Night)

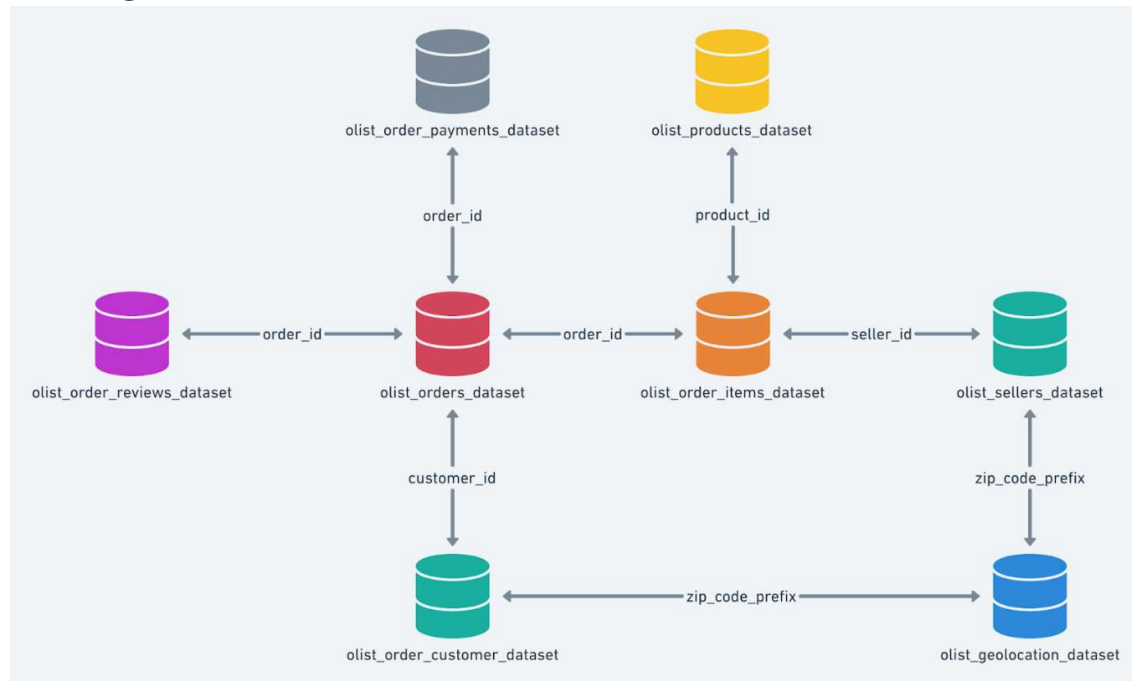
Context:

Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.

By analyzing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency.

ER Diagram:



Problem Statement:

Assuming I'm assigned the task of analysing the given dataset to extract valuable insights and provide actionable recommendations.

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset.

1.1 Data type of all columns in the "customers" table.

Query:

```
SELECT column_name, data_type
FROM scalar-dsml-sql-482016.Target.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = "customers";
```

Result:

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

1.2 Get the time range between which the orders were placed.

Query:

```
SELECT MIN(order_purchase_timestamp) AS First_order_date,
       MAX(order_purchase_timestamp) AS Latest_order_date
FROM `Target.orders`;
```

Result:

Row	First_order_date	Latest_order_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

1.3 Count the Cities & States of customers who ordered during the given period.

Query:

```
SELECT COUNT(DISTINCT c.customer_city) AS Total_Number_of_Cities,
       COUNT(DISTINCT c.customer_state) AS Total_Number_of_States
FROM `Target.orders` o JOIN `Target.customers` c
ON o.customer_id = c.customer_id;
```

Result:

Row	Total_Number_of_Cities	Total_Number_of_States
1	4119	27

2. In-depth Exploration.

2.1 Is there a growing trend in the no. of orders placed over the past years?

Query:

```
SELECT FORMAT_DATE('%Y-%m', order_purchase_timestamp) AS Year_Month,  
       COUNT(*) AS Orders_per_Month  
FROM `Target.orders`  
GROUP BY Year_Month  
ORDER BY Year_Month;
```

Result:

Row	Year_Month	Orders_per_Month
1	2016-09	4
2	2016-10	324
3	2016-12	1
4	2017-01	800
5	2017-02	1780
6	2017-03	2682
7	2017-04	2404
8	2017-05	3700
9	2017-06	3245
10	2017-07	4026

Chart:



Insights:

Yes, there is a clear growth trend in the number of orders placed. Orders grew rapidly through 2017 and plateaued in 2018, signaling a transition from high-growth to a maturity phase.

Recommendations:

Shift focus from acquisition to retention and repeat purchase strategies to sustain growth in a mature phase.

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Query:

```
SELECT EXTRACT(MONTH FROM order_purchase_timestamp) AS Month,  
       COUNT(*) AS Orders_Monthwise  
FROM `Target.orders`  
GROUP BY Month  
ORDER BY Month ASC;
```

Result:

Row	Month	Orders_Monthwise
1	1	8069
2	2	8508
3	3	9893
4	4	9343
5	5	10573
6	6	9412
7	7	10318
8	8	10843
9	9	4305
10	10	4959

Chart:



Insights:

Yes, there are indications of monthly seasonality. Orders dip noticeably in Sep–Oct after stable performance earlier in the year, indicating seasonal softness. So, there is some seasonal variation, but no consistent decline towards year end.

Recommendations:

Run targeted promotions and campaigns during Sep - Oct to smooth demand and reduce seasonal volatility.

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

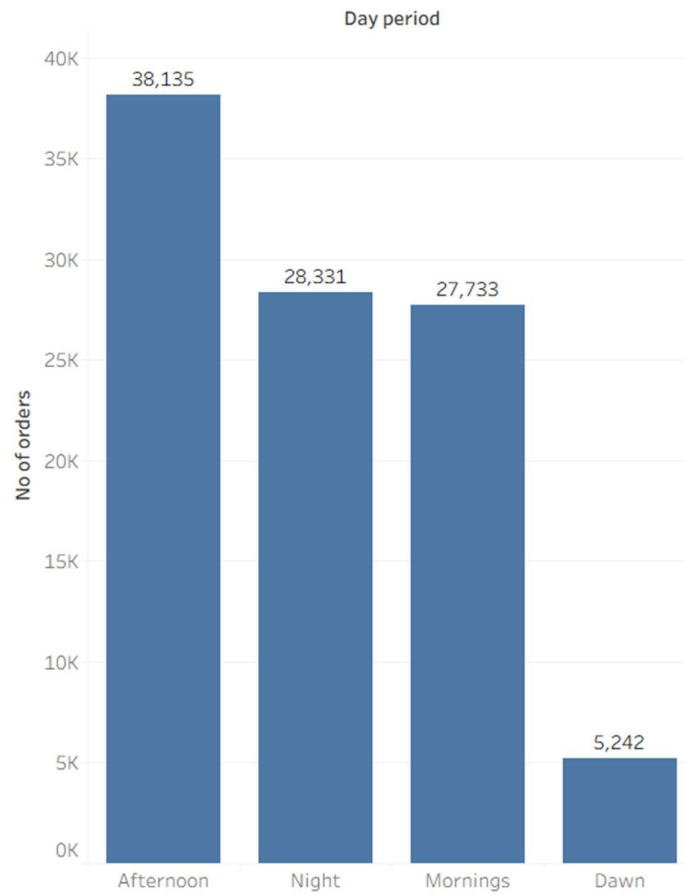
Query:

```
WITH Hour_Extract AS
(
SELECT EXTRACT(HOUR FROM order_purchase_timestamp) AS Hour_time
FROM `Target.orders`
)
SELECT CASE
    WHEN Hour_time BETWEEN 0 AND 6 THEN "Dawn"
    WHEN Hour_time BETWEEN 7 AND 12 THEN "Mornings"
    WHEN Hour_time BETWEEN 13 AND 18 THEN "Afternoon"
    WHEN Hour_time BETWEEN 19 AND 23 THEN "Night"
    END AS Day_period,
    COUNT(*) AS No_of_orders
FROM Hour_Extract
GROUP BY Day_period
ORDER BY No_of_orders DESC;
```

Result:

Row	Day_period	No_of_orders
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242

Chart:



Insights:

Brazilian customers mostly place their orders during the afternoon, between 1 PM and 6 PM, making it the most critical demand window.

Recommendations:

Schedule marketing pushes, flash sales, and operational readiness during peak afternoon hours to maximize conversions.

3. Evolution of E-commerce orders in the Brazil region.

3.1 Get the month on month no. of orders placed in each state.

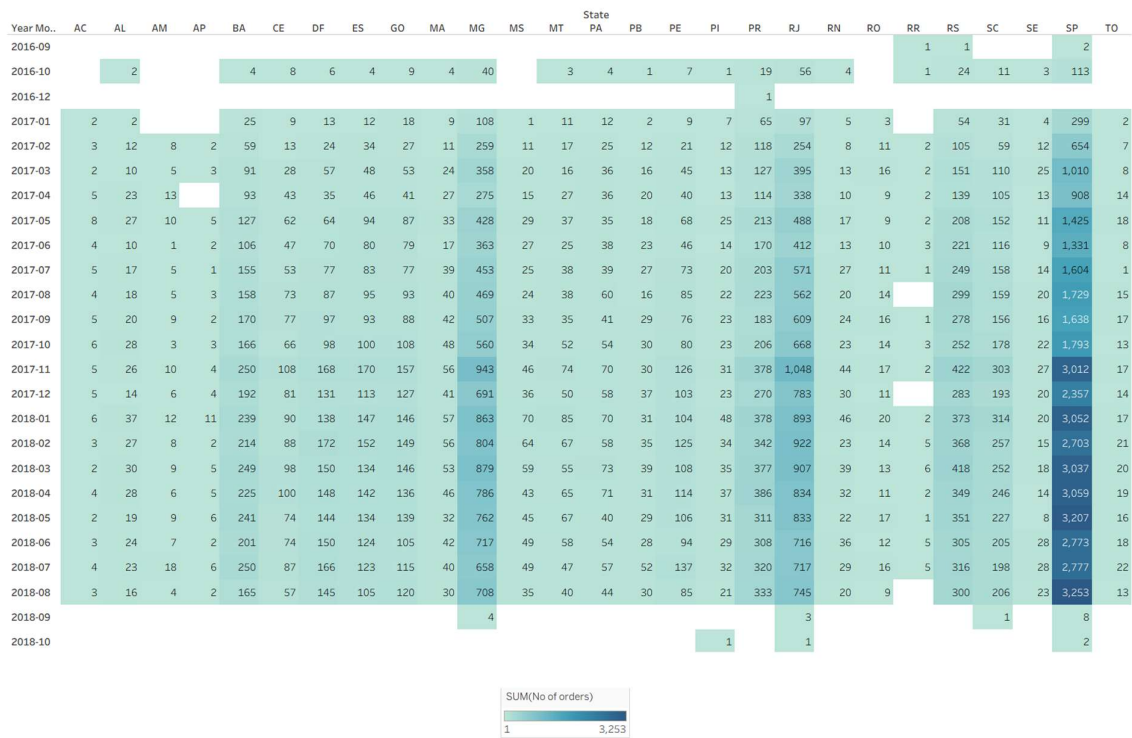
Query:

```
SELECT FORMAT_DATE("%Y-%m", o.order_purchase_timestamp) AS Year_Month,  
       c.customer_state AS State,  
       COUNT(*) AS No_of_orders  
FROM `Target.orders` o JOIN `Target.customers` c  
ON o.customer_id = c.customer_id  
GROUP BY Year_Month, State  
ORDER BY State ASC, Year_Month ASC;
```

Result:

Row	Year_Month	State	No_of_orders
1	2017-01	AC	2
2	2017-02	AC	3
3	2017-03	AC	2
4	2017-04	AC	5
5	2017-05	AC	8
6	2017-06	AC	4
7	2017-07	AC	5
8	2017-08	AC	4
9	2017-09	AC	5
10	2017-10	AC	6

Chart:



Insights:

Orders are highly concentrated in key states like SP, RJ, and MG, while mid-tier states are emerging as growth contributors. Smaller states show low order volumes and higher volatility.

Recommendations:

Double down on mid-tier states with targeted expansion while defending market share in top-performing states

3.2 How are the customers distributed across all the states?

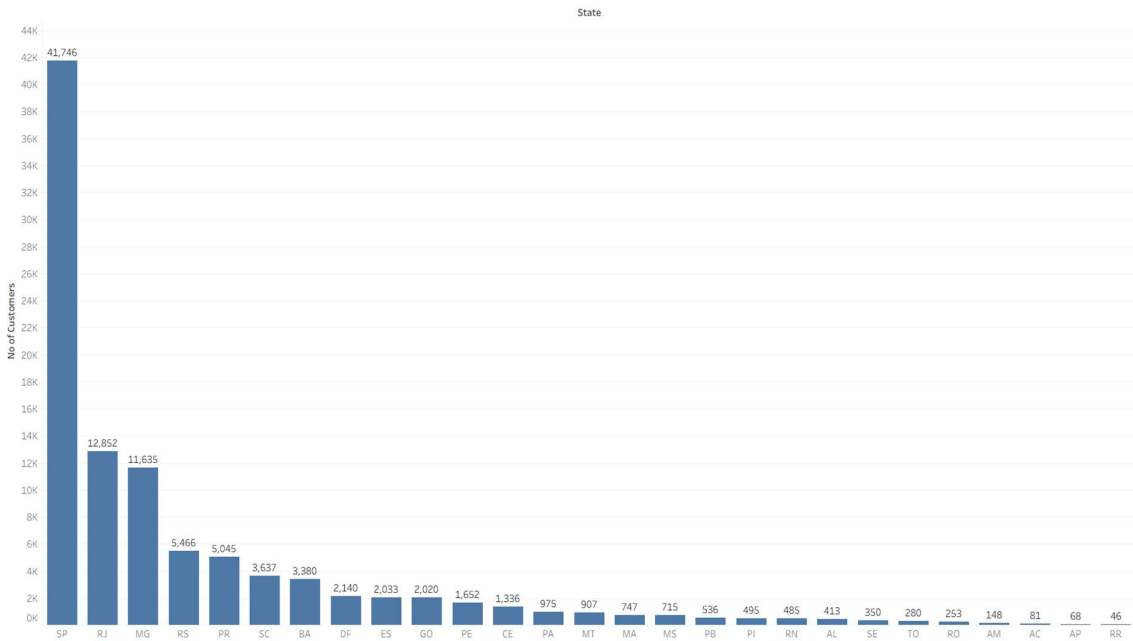
Query:

```
SELECT customer_state AS State,
       COUNT(DISTINCT customer_id) AS No_of_Customers
FROM `Target.customers`
GROUP BY State
ORDER BY No_of_Customers DESC;
```

Result:

Row	State	No_of_Customers
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

Chart:



Insights:

Customer distribution is highly skewed, with a significant concentration in SP, followed by a sharp drop across other states. The top few states contribute a large share of customers, while many smaller states form a long tail.

Recommendations:

Double down on mid-tier states with targeted expansion while defending market share in top-performing states

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only). You can use the "payment_value" column in the payments table to get the cost of orders.

Query:

```
WITH two_years AS
(
  SELECT order_id,
         EXTRACT(YEAR FROM order_purchase_timestamp) AS year
  FROM `Target.orders`
  WHERE EXTRACT(YEAR FROM order_purchase_timestamp) IN (2017, 2018) AND
         EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8
),
cost_per_year AS
(
  SELECT t.year,
         SUM(p.payment_value) AS total_cost
  FROM two_years t JOIN `Target.payments` p
  ON t.order_id = p.order_id
  GROUP BY t.year
)
SELECT year,
       ROUND(
         (total_cost - LAG(total_cost) OVER (ORDER BY year))
         * 100.0
         / LAG(total_cost) OVER (ORDER BY year), 2
       ) AS pct_increase
FROM cost_per_year;
```

Result:

Row	year	pct_increase
1	2017	null
2	2018	136.98

Chart:

No chart added, as objective was to calculate the % increase in the cost of orders from year 2017 to 2018 (Jan to Aug only), which was found to be 136.98%.

Insights:

A 136.98% increase in the cost of orders from 2017 to 2018 indicate a significant growth, with total order value more than doubling year-over-year. This reflects strong demand and business expansion, though such rapid growth may slow down over time.

Recommendations:

Sustain growth by scaling operations, improving cost efficiency and focus on customer retention.

4.2 Calculate the Total & Average value of order price for each state.

Query:

```
WITH cte AS
(
SELECT c.customer_state AS State,
       o.order_id AS o_id,
       SUM(oi.price) AS Total_price
FROM `Target.orders` o JOIN `Target.order_items` oi
ON o.order_id = oi.order_id
JOIN `Target.customers` c
ON o.customer_id = c.customer_id
GROUP BY c.customer_state, o.order_id
)

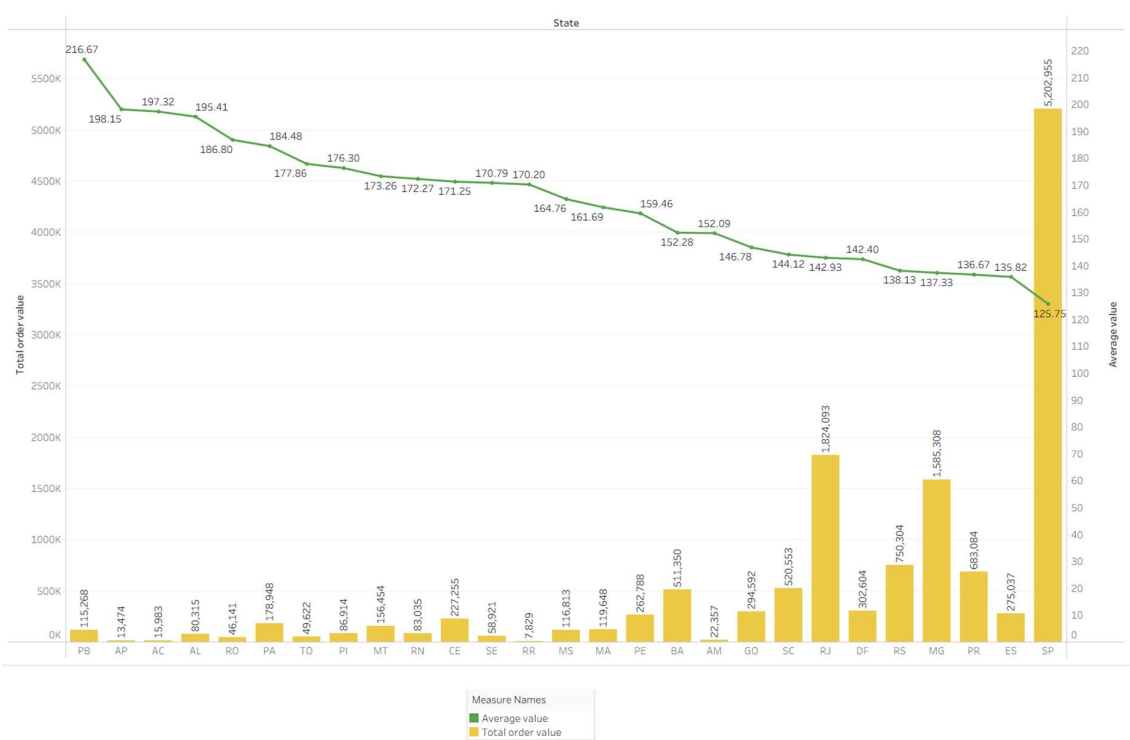
SELECT State,
       ROUND(SUM(Total_price),2) AS Total_order_value,
       ROUND(AVG(Total_price),2) AS Average_value
FROM cte
GROUP BY State
ORDER BY State;
```

-- For this analysis, order value has been calculated using the order items table instead of the payments table and therefore excludes freight charges.

Result:

Row	State	Total_order_value	Average_value
1	AC	15982.95	197.32
2	AL	80314.81	195.41
3	AM	22356.84	152.09
4	AP	13474.3	198.15
5	BA	511349.99	152.28
6	CE	227254.71	171.25
7	DF	302603.94	142.4
8	ES	275037.31	135.82
9	GO	294591.95	146.78
10	MA	119648.22	161.69

Chart:



Insights:

Revenue is highly concentrated in key states like SP, RJ, and MG driven mainly by higher order volumes rather than order value. Some smaller states show higher average order values, indicating premium customers, while mid-tier states present balanced performance and growth potential.

Recommendations:

Drive retention in top states, scale mid-tier states and target high-value customers in smaller states.

4.3 Calculate the Total & Average value of order freight for each state

Query:

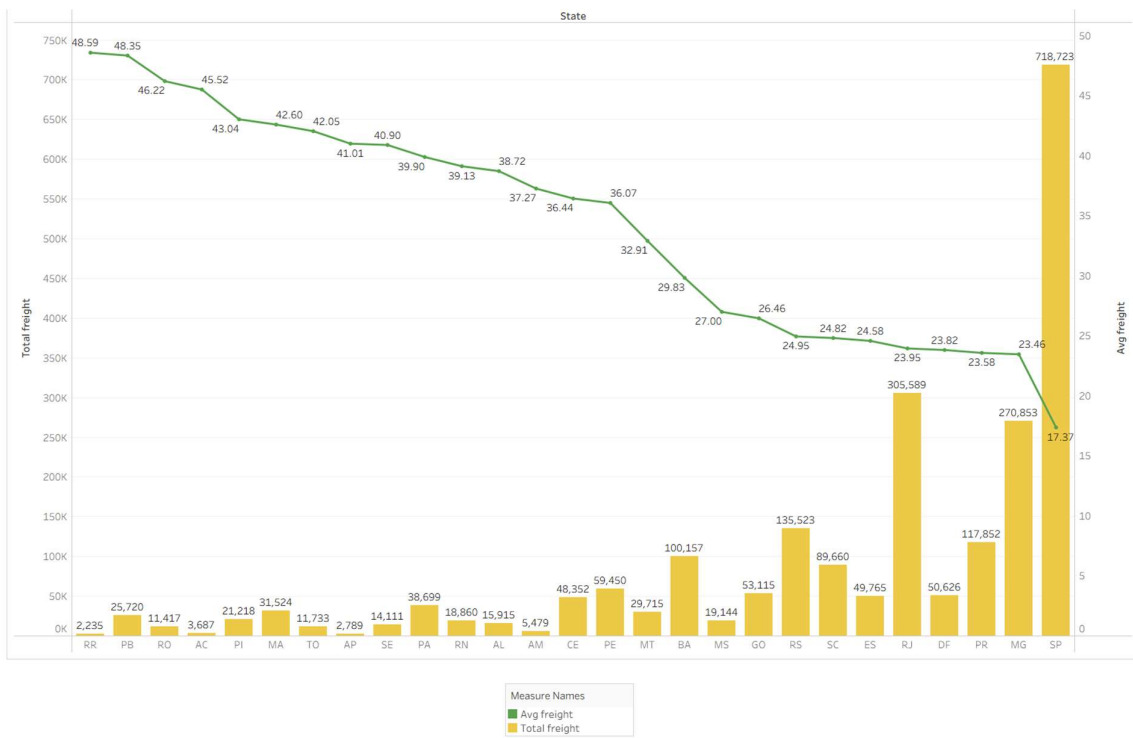
```
WITH cte AS
(
SELECT c.customer_state AS State,
       o.order_id,
       SUM(oi.freight_value) AS freight_sum
FROM `Target.orders` o JOIN `Target.order_items` oi
ON o.order_id = oi.order_id
JOIN `Target.customers` c
ON o.customer_id = c.customer_id
GROUP BY c.customer_state, o.order_id
)

SELECT State,
       ROUND(SUM(freight_sum),2) AS Total_freight,
       ROUND(AVG(freight_sum),2) AS Avg_freight
FROM cte
GROUP BY State
ORDER BY State;
```

Result:

Row	State	Total_freight	Avg_freight
1	AC	3686.75	45.52
2	AL	15914.59	38.72
3	AM	5478.89	37.27
4	AP	2788.5	41.01
5	BA	100156.68	29.83
6	CE	48351.59	36.44
7	DF	50625.5	23.82
8	ES	49764.6	24.58
9	GO	53114.98	26.46
10	MA	31523.77	42.6

Chart:



Insights:

Total freight is highest in major states due to higher order volumes, while smaller states have significantly higher average freight costs, indicating potential inefficiencies.

Recommendations:

Optimize logistics and delivery networks in smaller states to reduce costs. Maintain efficient operations in high-volume states.

5. Analysis based on sales, freight and delivery time.

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order

Query:

```
SELECT order_id,  
DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS  
time_to_deliver,  
DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS  
diff_estimated_delivery  
FROM `Target.orders`;
```

--There are NULL values for 2965 orders, which likely represent undelivered orders. Since the question doesn't explicitly ask to filter them out, I retained them. However, if the goal is to analyse only completed deliveries, we could add a WHERE condition to exclude NULL order_delivered_customer_date.

Result:

Row	order_id	time_to_deliver	diff_estimated_delivery
1	65d1e226dfaeb8cdc42f665422522d14	35	-16
2	2c45c33d2f9cb8ff8b1c86cc28c11c30	30	-28
3	1950d777989f6a877539f53795b4c3c3	30	12
4	bfb0f9bdef84302105ad712db648a6c	54	36
5	98974b076b01553d49ee6467905675a7	43	-6
6	c4b41c36dd589e901f6879f25a74ec1d	36	-14
7	d2292ff2201e74c5db154d1b7ae68cbb	29	-20
8	95e01270fcb9e9863423400103359279	30	-19
9	ed8c7b1b3eb256c70ce0c74231e1da88	44	-5
10	5cc475c7c03290048eb2e742cd64cb5e	68	18

Chart:

Since the question specifically asks for order-level calculations, I have not aggregated the data. However, for visualization or analysis purposes, aggregation would be useful to derive meaningful insights.

Insights:

The above query provides a base to analyse delivery performance including delays, average delivery time and the gap between estimated and actual delivery.

Recommendations:

Aggregate the data to identify patterns and improve delivery time accuracy. Focus on reducing delays and aligning estimates with actual performance.

5.2 Find out the top 5 states with the highest & lowest average freight value.

Query:

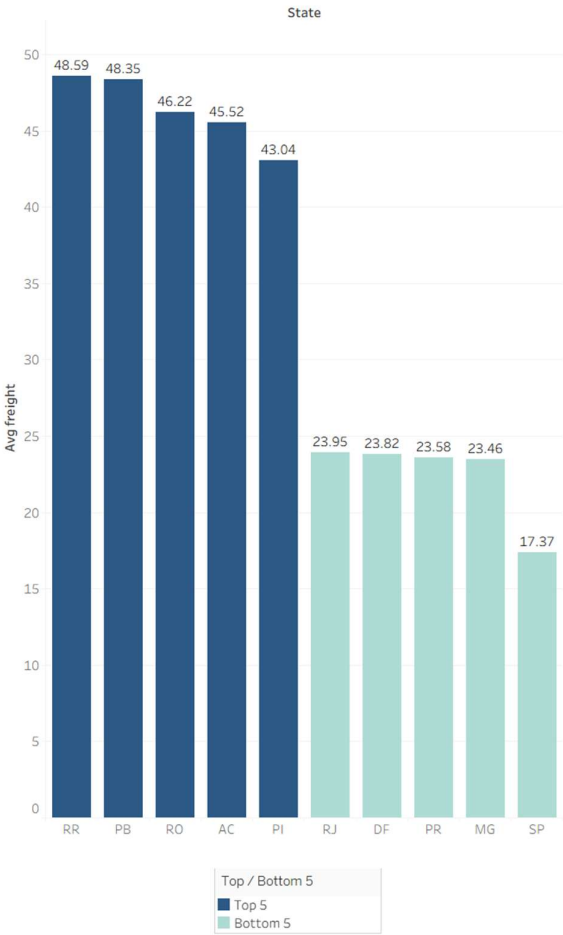
```
WITH cte AS
(
SELECT c.customer_state AS State,
       o.order_id,
       SUM(oi.freight_value) AS freight_sum
FROM `Target.orders` o JOIN `Target.order_items` oi
ON o.order_id = oi.order_id
JOIN `Target.customers` c
ON o.customer_id = c.customer_id
GROUP BY c.customer_state, o.order_id
),
Average_freight AS
(
SELECT State,
       ROUND(AVG(freight_sum),2) AS Avg_freight
FROM cte
GROUP BY State
),
Average_Ranked AS
(
SELECT State,
       Avg_freight,
       DENSE_RANK() OVER (ORDER BY Avg_freight DESC) AS Toprank,
       DENSE_RANK() OVER (ORDER BY Avg_freight ASC) AS Bottomrank
FROM Average_freight
)

SELECT State,
       Avg_freight
FROM Average_Ranked
WHERE Toprank <= 5
OR Bottomrank <= 5
ORDER BY Avg_freight ASC;
```

Result:

Row	State	Avg_freight
1	SP	17.37
2	MG	23.46
3	PR	23.58
4	DF	23.82
5	RJ	23.95
6	PI	43.04
7	AC	45.52
8	RO	46.22
9	PB	48.35
10	RR	48.59

Chart:



Insights:

Smaller states have disproportionately higher average freight costs, indicating logistics inefficiencies

Recommendations:

Optimize delivery routes or add regional hubs in high-cost states to reduce freight expenses.

5.3 Find out the top 5 states with the highest & lowest average delivery time.

Query:

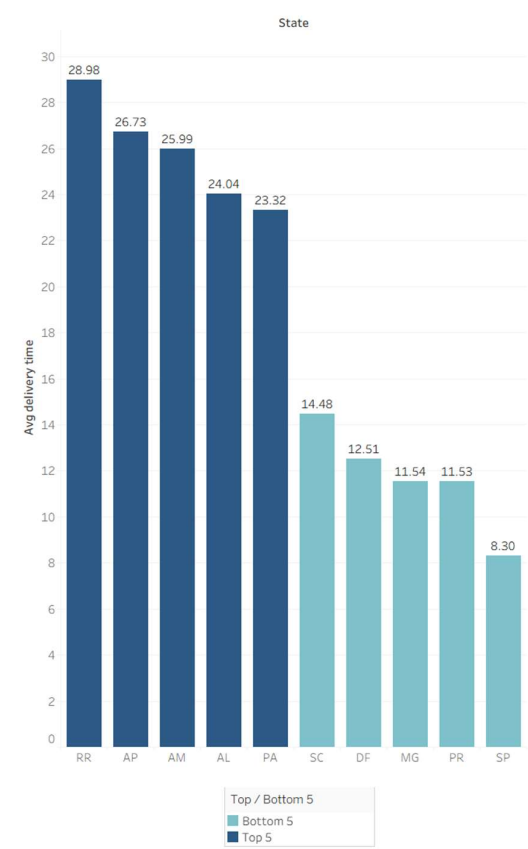
```
WITH delivery_time AS
(
SELECT order_id,
       customer_id,
       DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)
AS time_to_deliver
FROM `Target.orders`
WHERE order_delivered_customer_date IS NOT NULL
),
Average_time AS
(
SELECT c.customer_state AS State,
       AVG(d.time_to_deliver) AS Avg_time_to_deliver
FROM delivery_time d JOIN `Target.customers` c
ON d.customer_id = c.customer_id
GROUP BY c.customer_state
),
Ranked AS
(
SELECT State,
       Avg_time_to_deliver,
       DENSE_RANK() OVER (ORDER BY Avg_time_to_deliver ASC) Lowrank,
       DENSE_RANK() OVER (ORDER BY Avg_time_to_deliver DESC) Highrank
FROM Average_time
)

SELECT State,
       ROUND(Avg_time_to_deliver,2) AS Avg_delivery_time
FROM Ranked
WHERE Lowrank <= 5 OR
Highrank <= 5
ORDER BY Avg_delivery_time ASC;
```

Result:

Row	State	Avg_delivery_time
1	SP	8.3
2	PR	11.53
3	MG	11.54
4	DF	12.51
5	SC	14.48
6	PA	23.32
7	AL	24.04
8	AM	25.99
9	AP	26.73
10	RR	28.98

Chart:



Insights:

There is a significant disparity in delivery times across states, States like RR and AP are taking almost 3 times longer than SP, showing clear regional inefficiency in deliveries.

Recommendations:

Improve delivery speed in high-delay states by adding local hubs and better route planning using SP as the benchmark.

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery. You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Query:

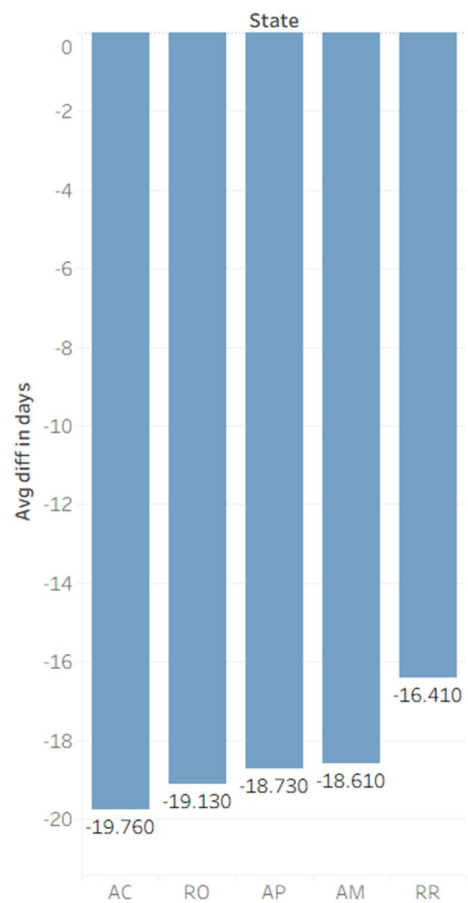
```
WITH Difference AS
(
SELECT order_id,
       customer_id,
       DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY)
AS diff_estimated_delivery
FROM `Target.orders`
WHERE order_delivered_customer_date IS NOT NULL
)

SELECT c.customer_state AS State,
       ROUND(AVG(d.diff_estimated_delivery),2) AS Avg_diff_in_days
FROM Difference d JOIN `Target.customers` c
ON d.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY Avg_diff_in_days ASC
LIMIT 5;
```

Result:

Row	State	Avg_diff_in_days
1	AC	-19.76
2	RO	-19.13
3	AP	-18.73
4	AM	-18.61
5	RR	-16.41

Chart:



Insights:

All states show negative values, indicating deliveries are consistently happening earlier than estimated, likely due to a large buffer in estimated delivery dates.

Recommendations:

The above chart shows a narrow range of 16–20 days for the top 5 states, suggesting minimal variation across them. This may be due to systematic overestimation, which can be optimized to set more accurate expectations and improve customer experience.

6. Analysis based on the payments

6.1 Find the month on month no. of orders placed using different payment types.

Query:

```
SELECT FORMAT_DATE('%Y-%m',o.order_purchase_timestamp) AS Year_month,  
       p.payment_type,  
       COUNT(DISTINCT o.order_id) AS No_of_orders  
FROM `Target.orders` o JOIN `Target.payments` p  
ON o.order_id = p.order_id  
GROUP BY Year_month, p.payment_type  
ORDER BY Year_month ASC, UPPER(p.payment_type) ASC;
```

Result:

Row	Year_month	payment_type	No_of_orders
1	2016-09	credit_card	3
2	2016-10	credit_card	254
3	2016-10	debit_card	2
4	2016-10	UPI	63
5	2016-10	voucher	23
6	2016-12	credit_card	1
7	2017-01	credit_card	583
8	2017-01	debit_card	9
9	2017-01	UPI	197
10	2017-01	voucher	61

Chart:

Year month	Payment Type				
	credit_card	debit_card	UPI	voucher	not_defined
2016-09	3				
2016-10	254	2	63	23	
2016-12	1				
2017-01	583	9	197	61	
2017-02	1,356	13	398	119	
2017-03	2,016	31	590	200	
2017-04	1,846	27	496	202	
2017-05	2,853	30	772	289	
2017-06	2,463	27	707	239	
2017-07	3,086	22	845	364	
2017-08	3,284	34	938	294	
2017-09	3,283	43	903	287	
2017-10	3,524	52	993	291	
2017-11	5,897	70	1,509	387	
2017-12	4,377	64	1,160	294	
2018-01	5,520	109	1,518	416	
2018-02	5,253	69	1,325	305	
2018-03	5,691	78	1,352	391	
2018-04	5,455	97	1,287	370	
2018-05	5,497	51	1,263	324	
2018-06	4,813	182	1,100	324	
2018-07	4,755	242	1,229	281	
2018-08	4,985	277	1,139	295	2
2018-09				15	1
2018-10				4	



Insights:

Credit cards dominate order payments throughout the period, while UPI shows a clear upward trend, indicating growing digital adoption. Debit card usage remains minimal and vouchers contribute steadily, likely driven by promotions. The 'Not defined' category is negligible, so doesn't impact our analysis.

Recommendations:

Accelerate UPI adoption through targeted incentives to reduce reliance on credit cards.

6.2 Find the no. of orders placed on the basis of the payment installments that have been paid.

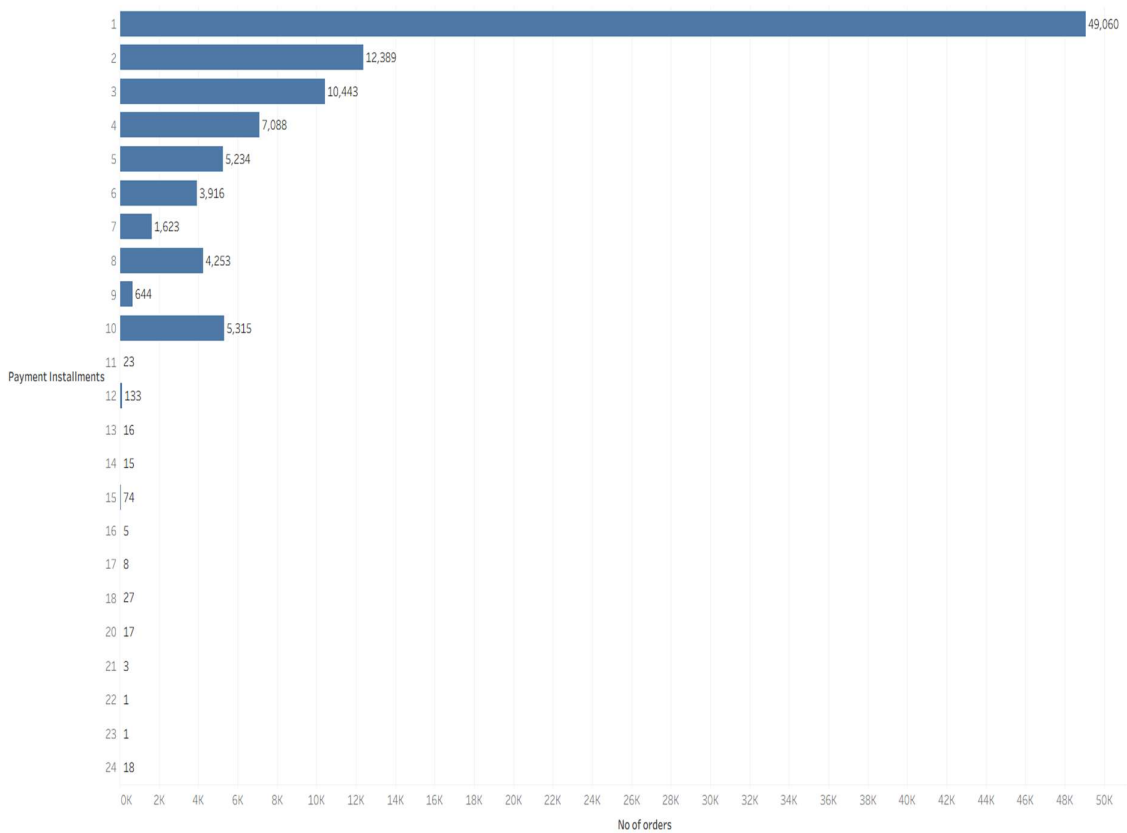
Query:

```
SELECT payment_installments,  
       COUNT(DISTINCT order_id) AS No_of_orders  
FROM `Target.payments`  
WHERE payment_installments > 0  
GROUP BY payment_installments  
ORDER BY payment_installments;
```

Result:

Row	payment_installments	No_of_orders
1	1	49060
2	2	12389
3	3	10443
4	4	7088
5	5	5234
6	6	3916
7	7	1623
8	8	4253
9	9	644
10	10	5315

Chart:



Insights:

Most customers prefer single-payment transactions, with a sharp drop in usage as installments increase. While short-term EMI options (2–6 months) are moderately used, there are small spikes at standard plans like 8 and 10 months. Beyond 10 installments, usage is minimal, indicating low preference for long-term financing.

Recommendations:

Continue prioritizing single-payment flows while promoting short-term EMI options (3–6 months) over long-term plans.

Project conclusion

Target Brazil shows strong growth transitioning into a maturity phase, with orders peaking in 2017 and stabilizing in 2018. Demand is concentrated in the afternoon, with a clear seasonal dip in Sep–Oct.

Geographically, revenue is driven by high-volume states like SP, while smaller states show higher costs and untapped potential. Despite overall strong delivery performance (consistently beating estimates), a 3 times gap between best and worst states highlights regional inefficiencies in logistics.

Regarding payments credit cards dominate, but rising UPI adoption signals a shift toward digital, while customers clearly prefer single payments and short-term EMIs.

Priority actions: Strengthen retention in top states, expand in mid-tier markets, fix logistics gaps in high-delay regions and accelerate UPI adoption.